

C# 프로그래밍

동명대학교 게임학부 · 2026년 1학기

Lecture 7

C# 그래픽과 슈팅 게임

Windows Forms로 배우는 단계별 게임 개발

강영민

동명대학교 게임학부

Contents

1 강의 개요	2
1.1 학습 목표	2
1.2 강의 구성	2
1.3 프로젝트 구조	2
2 기본 그래픽: Step 1-3	4
2.1 Step 1: 점 찍기 — Form과 Graphics의 기초	4
2.2 Step 2: 선 그리기 — Pen과 DrawLine	5
2.3 Step 3: 도형 그리기 — 사각형, 원, 다각형	6
3 애니메이션과 입력: Step 4-5	8
3.1 Step 4: 애니메이션 — Timer와 Invalidate	8
3.2 Step 5: 키보드 입력 — OnKeyDown / OnKeyUp	9
4 클래스 도입: Step 6-7	12
4.1 Step 6: Player 클래스 — 비행기 만들기	12
4.2 Step 7: Bullet 클래스 — 총알 발사	14
5 슈팅 게임 완성: Step 8-10	17
5.1 Step 8: Enemy 클래스 — 적 등장	17
5.2 Step 9: Hud 클래스 — 점수와 생명 표시	20
5.3 Step 10: 슈팅 게임 완성	20
6 종합 정리	25
6.1 전체 클래스 구조	25
6.2 단계별 파일 구성 변화	25
6.3 이번 강의에서 배운 핵심 개념	26
6.4 게임 루프 패턴 정리	26

1. 강의 개요

1.1. 학습 목표

이번 강의에서는 Windows Forms의 그래픽 기능을 활용하여, 점 하나 찍기부터 슈팅 게임 완성까지 10단계로 진행한다.

- Form, Graphics 클래스를 사용하여 화면에 그래픽을 출력할 수 있다.
- Timer를 이용한 게임 루프(애니메이션)의 원리를 이해한다.
- 키보드 입력을 처리하여 화면의 객체를 제어할 수 있다.
- 게임 요소(플레이어, 총알, 적, HUD)를 각각 **클래스로 분리**하여 설계한다.
- 클래스를 **별도 파일**로 나누어 프로젝트를 구성하는 방법을 익힌다.

1.2. 강의 구성

파트	Step	내용
기본 그래픽	Step 1-3	점, 선, 도형 그리기
애니메이션과 입력	Step 4-5	Timer 루프, 키보드 입력
클래스 도입	Step 6-7	Player 클래스, Bullet 클래스
슈팅 게임 완성	Step 8-10	Enemy, HUD, 최종 게임

1.3. 프로젝트 구조

각 Step은 독립된 프로젝트이며, Step이 진행될수록 클래스 파일이 추가된다.

파일 구성의 변화

Step01~05/	Step06/	Step10/
Program.cs	Program.cs	Program.cs
GameForm.cs	GameForm.cs	GameForm.cs
	Player.cs	Player.cs
		Bullet.cs
		Enemy.cs
		Hud.cs

모든 프로젝트에서 Program.cs는 동일하다:

```

1 class Program
2 {
3     static void Main()
4     {
5         Application.Run(new GameForm());
6     }
7 }
```

Application.Run()이란?

Windows GUI 프로그램은 운영체제로부터 이벤트(마우스, 키보드, 화면 그리기 등)를 계속 받아 처리하는 **메시지 루프**가 필요하다. `Application.Run(new GameForm())`은 다음 세 가지를 수행한다:

1. `GameForm` 객체를 생성하고 화면에 표시
2. 메시지 루프를 시작하여 이벤트를 계속 수신
3. 사용자가 창을 닫으면 루프를 종료하고 프로그램 종료

이 함수가 없으면 창이 열리자마자 바로 닫힌다.

프로젝트 파일(.csproj)은 왜 필요한가?

C# 코드만으로는 `Form`, `Graphics` 등의 Windows Forms 클래스를 사용할 수 없다. 이 클래스들은 .NET 기본 라이브러리가 아니라 **별도의 Windows 전용 라이브러리**에 들어 있기 때문이다.

.csproj 파일은 “이 프로젝트가 어떤 라이브러리를 사용하는지”를 컴파일러에게 알려주는 **설정 파일**이다.

```

1 <Project Sdk="Microsoft.NET.Sdk">
2   <PropertyGroup>
3     <OutputType>WinExe</OutputType>
4     <TargetFramework>net10.0-windows</TargetFramework>
5     <UseWindowsForms>true</UseWindowsForms>
6   </PropertyGroup>
7 </Project>

```

각 항목의 역할:

설정	역할
<code>OutputType: WinExe</code>	콘솔 창 없이 Windows 응용 프로그램으로 실행
<code>TargetFramework</code>	대상 .NET 버전 (Windows 전용 API 포함)
<code>UseWindowsForms: true</code>	<code>System.Windows.Forms</code> , <code>System.Drawing</code> 참조 추가

이 파일이 없거나 `UseWindowsForms`가 빠지면, 컴파일러는 `Form`이 무엇인지 모르기 때문에 다음과 같은 에러가 발생한다:

```

error CS0234: The type or namespace name 'Forms'
does not exist in the namespace 'System.Windows'

```

비유 .cs 파일이 “레시피(코드)”라면, .csproj 파일은 “재료 목록(어떤 라이브러리를 쓸 것인가)”이다. 재료 목록 없이 레시피만으로는 요리를 완성할 수 없다.

2. 기본 그래픽: Step 1-3

2.1. Step 1: 점 찍기 — Form과 Graphics의 기초

화면에 그래픽을 그리려면 두 가지가 필요하다:

1. **Form** — 그림을 그릴 **창**(윈도우)
2. **Graphics** — 그림을 그리는 **도구**(펜, 붓)

GameForm.cs

```

1 using System;
2 using System.Drawing;
3 using System.Windows.Forms;
4
5 class GameForm : Form
6 {
7     public GameForm()
8     {
9         Text = "Step 1 - Point";
10        ClientSize = new Size(800, 600);
11        BackColor = Color.Black;
12        DoubleBuffered = true;
13    }
14
15    protected override void OnPaint(PaintEventArgs e)
16    {
17        Graphics g = e.Graphics;
18
19        // Draw a single point (2x2 rectangle)
20        g.FillRectangle(Brushes.White, 400, 300, 2, 2);
21
22        // Draw a circle using many points
23        for (int i = 0; i < 360; i += 5)
24        {
25            double rad = i * Math.PI / 180;
26            int px = 400 + (int)(100 * Math.Cos(rad));
27            int py = 300 + (int)(100 * Math.Sin(rad));
28            g.FillRectangle(Brushes.Cyan, px, py, 2, 2);
29        }
30    }
31 }
```

핵심 개념: Form 상속과 OnPaint

- `class GameForm : Form` — Form을 상속받아 나만의 폼을 만든다.
- 생성자에서 창 크기, 배경색, 제목 등을 설정한다.
- `DoubleBuffered = true` — 화면 깜빡임을 방지한다. 이 속성은 `protected`이므로 상속받은 클래스에서만 설정할 수 있다.

- `OnPaint()` — 화면을 그려야 할 때 자동으로 호출되는 메서드이다. 이 메서드를 `override`하여 우리가 원하는 그림을 그린다.
- `Graphics g` — 모든 그리기 명령은 이 객체를 통해 수행한다.
- `FillRectangle(Brushes.White, x, y, w, h)` — (x, y) 위치에 $w \times h$ 크기의 채워진 사각형을 그린다. 크기를 2×2 로 하면 “점”처럼 보인다.

protected override의 의미

`OnPaint`의 선언을 분해하면 다음과 같다:

```
protected override void OnPaint(PaintEventArgs e)
```

override — 왜 필요한가? `Form` 클래스에는 `OnPaint`가 `virtual`(재정의 가능)로 선언되어 있다. `Windows`가 “화면을 다시 그려라”라고 요청하면 `Form`은 자신의 `OnPaint`를 호출한다.

- `override`를 붙이면 — 부모의 `OnPaint` 대신 우리 코드가 실행된다.
- `override`가 없으면 — 이름만 같은 별개의 메서드가 되어, `Windows`가 호출하는 것은 여전히 부모의 빈 `OnPaint`이다. **결과: 화면에 아무것도 그려지지 않는다.**

protected — 왜 필요한가? `protected`는 “이 클래스와 자식 클래스만 사용 가능”이라는 접근 제한자이다. 부모 `Form`이 `OnPaint`를 `protected virtual`로 선언했으므로, 재정의할 때도 동일한 접근 수준을 유지해야 한다.

접근 제한자	의미	<code>OnPaint</code> 에 사용 가능?
<code>public</code>	어디서든 접근 가능	O (범위를 넓히는 것은 허용)
<code>protected</code>	자신과 자식 클래스만 접근	O (부모와 동일)
<code>private</code>	자신만 접근 가능	X (컴파일 에러)

C#에서 메서드를 `override`할 때, 부모보다 접근 범위를 좁힐 수 없다. `public`으로 넓히는 것은 가능하지만, 관례적으로 부모와 같은 `protected`를 사용한다.

2.2. Step 2: 선 그리기 — Pen과 DrawLine

GameForm.cs

```

1 class GameForm : Form
2 {
3     public GameForm()
4     {
5         Text = "Step 2 - Line";
6         ClientSize = new Size(800, 600);
7         BackColor = Color.Black;
8         DoubleBuffered = true;
9     }

```

```

10
11     protected override void OnPaint(PaintEventArgs e)
12     {
13         Graphics g = e.Graphics;
14
15         // Basic line
16         g.DrawLine(Pens.Red, 100, 100, 700, 100);
17
18         // Diagonal line
19         g.DrawLine(Pens.Green, 100, 200, 700, 400);
20
21         // Thick line (create Pen object)
22         Pen thickPen = new Pen(Color.Yellow, 3);
23         g.DrawLine(thickPen, 100, 500, 700, 500);
24
25         // Grid
26         Pen gridPen = new Pen(
27             Color.FromArgb(60, 255, 255, 255));
28         for (int x = 0; x < 800; x += 50)
29             g.DrawLine(gridPen, x, 0, x, 600);
30         for (int y = 0; y < 600; y += 50)
31             g.DrawLine(gridPen, 0, y, 800, y);
32     }
33 }

```

Pen vs Brush

	Pen	Brush
용도	선/외곽선 그리기	면 채우기
메서드	DrawLine, DrawRectangle	FillRectangle, FillEllipse
기본 제공	Pens.Red	Brushes.Red
커스텀	new Pen(Color, width)	new SolidBrush(Color)

Color.FromArgb(60, 255, 255, 255)는 반투명 흰색이다 (알파값 60/255).

2.3. Step 3: 도형 그리기 — 사각형, 원, 다각형

GameForm.cs의 OnPaint 메서드

```

1     protected override void OnPaint(PaintEventArgs e)
2     {
3         Graphics g = e.Graphics;
4
5         // Rectangle: outline only
6         g.DrawRectangle(Pens.Red, 50, 50, 200, 120);
7
8         // Rectangle: filled
9         g.FillRectangle(Brushes.Blue, 300, 50, 200, 120);
10

```

```

11 // Circle: outline only
12 g.DrawEllipse(Pens.Green, 50, 250, 150, 150);
13
14 // Circle: filled
15 g.FillEllipse(Brushes.Magenta, 250, 250, 150, 150);
16
17 // Triangle (polygon)
18 PointF[] triangle = new PointF[]
19 {
20     new PointF(550, 50),
21     new PointF(450, 200),
22     new PointF(650, 200)
23 };
24 g.FillPolygon(Brushes.Cyan, triangle);
25
26 // Pentagon (polygon)
27 PointF[] pentagon = new PointF[]
28 {
29     new PointF(550, 280), new PointF(490, 340),
30     new PointF(510, 420), new PointF(590, 420),
31     new PointF(610, 340)
32 };
33 g.FillPolygon(Brushes.Orange, pentagon);
34 }

```

Graphics 그리기 메서드 요약

메서드	설명
FillRectangle(brush, x, y, w, h)	채워진 사각형
DrawRectangle(pen, x, y, w, h)	사각형 외곽선
FillEllipse(brush, x, y, w, h)	채워진 타원 (w=h이면 원)
DrawEllipse(pen, x, y, w, h)	타원 외곽선
FillPolygon(brush, PointF[])	채워진 다각형
DrawLine(pen, x1, y1, x2, y2)	직선
DrawString(text, font, brush, x, y)	텍스트 출력

실습 1: 기본 그래픽

1. Step01-03을 각각 dotnet run으로 실행해 보자.
2. Step03에서 삼각형의 좌표를 바꿔 위치를 이동시켜 보자.
3. 새로운 도형(별, 육각형 등)을 FillPolygon으로 그려 보자.

3. 애니메이션과 입력: Step 4-5

3.1. Step 4: 애니메이션 — Timer와 Invalidate

지금까지는 정지된 그림만 그렸다. 게임을 만들려면 “매 프레임마다 위치를 바꾸고 다시 그리는” 게임 루프가 필요하다.

게임 루프의 원리

1. Timer가 일정 간격(16ms $\approx$ 60fps)으로 OnUpdate을 호출
2. OnUpdate에서 게임 상태(위치, 속도 등)를 갱신
3. Invalidate()를 호출하면 Windows가 OnPaint를 다시 호출
4. OnPaint에서 갱신된 상태로 화면을 다시 그림

이 과정이 초당 약 60번 반복되면서 “움직이는” 것처럼 보인다.

GameForm.cs

```
1  class GameForm : Form
2  {
3      float x = 100;
4      float y = 100;
5      float dx = 4;
6      float dy = 3;
7      Timer timer;
8
9      public GameForm()
10     {
11         Text = "Step 4 - Animation";
12         ClientSize = new Size(800, 600);
13         BackColor = Color.Black;
14         DoubleBuffered = true;
15
16         // Game loop timer (~60fps)
17         timer = new Timer();
18         timer.Interval = 16;
19         timer.Tick += OnUpdate;
20         timer.Start();
21     }
22
23     void OnUpdate(object sender, EventArgs e)
24     {
25         // Update position
26         x += dx;
27         y += dy;
28
29         // Bounce off walls
30         if (x < 0 || x > 760) dx = -dx;
31         if (y < 0 || y > 560) dy = -dy;
```

```

32         Invalidate(); // Request repaint
33     }
34
35     protected override void OnPaint(PaintEventArgs e)
36     {
37         e.Graphics.FillEllipse(Brushes.Cyan,
38             x, y, 40, 40);
39     }
40 }
41

```

timer.Tick += OnUpdate; 의 의미

이 한 줄은 **이벤트(event)**와 **메서드**를 연결하는 코드이다. 각 부분의 의미는 다음과 같다:

코드	의미
timer.Tick	Timer가 Interval마다 발생시키는 이벤트
+=	이벤트에 메서드를 등록(구독) 하는 연산자
OnUpdate	이벤트가 발생할 때 실행될 메서드

즉, “타이머가 틱할 때마다 OnUpdate를 호출해라”라는 뜻이다.

왜 OnUpdate() 가 아니라 OnUpdate 인가? 괄호가 없는 이유는 **지금 호출하는 것이 아니라 나중에 호출할 메서드의 이름을 넘기는 것**이기 때문이다.

- OnUpdate() — 지금 즉시 실행하고 **결과 값**을 전달
- OnUpdate — 메서드 자체를 **참조**로 전달 (나중에 호출됨)

내부적으로 일어나는 일 C#에서 메서드를 변수처럼 저장할 수 있는 타입을 **delegate(대리자)**라고 한다. 위 코드는 컴파일러가 다음과 같이 변환한다:

```
timer.Tick += new EventHandler(OnUpdate);
```

EventHandler는 .NET에 미리 정의된 delegate 타입이며, void 메서드(object sender, EventArgs e) 형태의 메서드를 저장할 수 있다. 그래서 OnUpdate의 매개변수가 (object sender, EventArgs e)인 것이다.

핵심: Invalidate()의 역할

Invalidate()는 “화면이 바뀌었으니 다시 그려달라”고 Windows에 요청하는 것이다. 이 호출이 있어야 OnPaint가 새로운 좌표로 다시 실행된다. 직접 OnPaint를 호출하는 것이 아니라, **Windows에게 요청**하는 방식이다. 이렇게 하면 Windows가 적절한 타이밍에 화면을 갱신해 준다.

3.2. Step 5: 키보드 입력 — OnKeyDown / OnKeyUp

사용자의 키보드 입력에 따라 도형을 움직이게 한다.

HashSet<Keys>란?

HashSet<T>는 **중복 없이** 값을 저장하는 컬렉션이다. 같은 값을 여러 번 Add해도 하나만 저장된다.

컬렉션	중복 허용	주요 용도
List<T>	O	순서가 있는 목록 (총알, 적 등)
HashSet<T>	X	포함 여부만 중요한 집합 (현재 눌린 키)

키보드 입력에 HashSet을 쓰는 이유 키를 누르고 있으면 OnKeyDown이 반복 호출된다. List를 쓰면 같은 키가 여러 번 추가되지만, HashSet은 이미 있는 값이면 무시하므로 항상 “현재 눌린 키 목록”을 정확히 유지한다.

```
HashSet<Keys> keys = new HashSet<Keys>();

keys.Add(Keys.Left);           // {Left}
keys.Add(Keys.Left);           // {Left} -- no duplicate
keys.Add(Keys.Up);             // {Left, Up}
keys.Contains(Keys.Left);      // true
keys.Remove(Keys.Left);        // {Up}
```

GameForm.cs

```
1 class GameForm : Form
2 {
3     float x = 380;
4     float y = 280;
5     float speed = 5;
6     HashSet<Keys> keys = new HashSet<Keys>();
7     Timer timer;
8
9     public GameForm()
10    {
11        Text = "Step 5 - Input";
12        ClientSize = new Size(800, 600);
13        BackColor = Color.Black;
14        DoubleBuffered = true;
15
16        timer = new Timer();
17        timer.Interval = 16;
18        timer.Tick += OnUpdate;
19        timer.Start();
20    }
21
22    void OnUpdate(object sender, EventArgs e)
23    {
24        if (keys.Contains(Keys.Left)) x -= speed;
25        if (keys.Contains(Keys.Right)) x += speed;
26        if (keys.Contains(Keys.Up)) y -= speed;
```

```

27         if (keys.Contains(Keys.Down)) y += speed;
28
29         x = Math.Clamp(x, 0, 760);
30         y = Math.Clamp(y, 0, 560);
31
32         Invalidate();
33     }
34
35     protected override void OnKeyDown(KeyEventArgs e)
36     {
37         keys.Add(e.KeyCode);
38     }
39
40     protected override void OnKeyUp(KeyEventArgs e)
41     {
42         keys.Remove(e.KeyCode);
43     }
44
45     protected override void OnPaint(PaintEventArgs e)
46     {
47         e.Graphics.FillRectangle(Brushes.Lime,
48             x, y, 40, 40);
49     }
50 }

```

왜 HashSet을 사용하는가?

OnKeyDown은 키를 누를 때 한 번 호출되고, OnKeyUp은 키를 뺄 때 한 번 호출된다. HashSet<Keys>에 현재 눌린 키를 저장하면:

- **동시 입력** 처리 가능 — 위+오른쪽을 동시에 누르면 대각선 이동
- **연속 이동** 가능 — 키를 누르고 있는 동안 매 프레임마다 이동

Math.Clamp(x, 0, 760)은 x 값을 0 이상, 760 이하로 제한하여 화면 밖으로 나가지 않도록 한다.

실습 2: 애니메이션과 입력

1. Step04를 실행하고, 공이 벽에서 반사되는 것을 확인하자.
2. Step04에서 dx, dy 값을 바꿔 속도를 조절해 보자.
3. Step05를 실행하고, 방향키로 사각형을 움직여 보자.
4. Step05에서 speed 값을 바꿔 이동 속도를 변경해 보자.

4. 클래스 도입: Step 6–7

지금까지 GameForm 안에 모든 코드가 있었다. 이제 게임 요소를 **클래스로 분리**하여 별도 파일에 담는다.

4.1. Step 6: Player 클래스 — 비행기 만들기

키보드 이동 로직과 그리기 로직을 Player 클래스로 분리한다.

Player.cs — 새 파일

```
1 using System;
2 using System.Collections.Generic;
3 using System.Drawing;
4 using System.Windows.Forms;
5
6 class Player
7 {
8     public float X;
9     public float Y;
10    float speed = 5;
11
12    public Player(float x, float y)
13    {
14        X = x;
15        Y = y;
16    }
17
18    public void Update(HashSet<Keys> keys)
19    {
20        if (keys.Contains(Keys.Left)) X -= speed;
21        if (keys.Contains(Keys.Right)) X += speed;
22        if (keys.Contains(Keys.Up)) Y -= speed;
23        if (keys.Contains(Keys.Down)) Y += speed;
24        X = Math.Clamp(X, 15, 785);
25        Y = Math.Clamp(Y, 20, 580);
26    }
27
28    public void Draw(Graphics g)
29    {
30        // Airplane body (triangle)
31        PointF[] body = new PointF[]
32        {
33            new PointF(X, Y - 20),
34            new PointF(X - 15, Y + 15),
35            new PointF(X + 15, Y + 15)
36        };
37        g.FillPolygon(Brushes.Cyan, body);
38
39        // Wings
40        g.FillRectangle(Brushes.DodgerBlue,
```

```
41         X - 25, Y + 5, 50, 6);
42     }
43 }
```

GameForm.cs — Player를 사용

```
1  class GameForm : Form
2  {
3      Player player = new Player(400, 520);
4      HashSet<Keys> keys = new HashSet<Keys>();
5      Timer timer;
6
7      public GameForm()
8      {
9          Text = "Step 6 - Player Class";
10         ClientSize = new Size(800, 600);
11         BackColor = Color.Black;
12         DoubleBuffered = true;
13
14         timer = new Timer();
15         timer.Interval = 16;
16         timer.Tick += OnUpdate;
17         timer.Start();
18     }
19
20     void OnUpdate(object sender, EventArgs e)
21     {
22         player.Update(keys);
23         Invalidate();
24     }
25
26     protected override void OnKeyDown(KeyEventArgs e)
27     {
28         keys.Add(e.KeyCode);
29     }
30
31     protected override void OnKeyUp(KeyEventArgs e)
32     {
33         keys.Remove(e.KeyCode);
34     }
35
36     protected override void OnPaint(PaintEventArgs e)
37     {
38         player.Draw(e.Graphics);
39     }
40 }
```

Step 5 vs Step 6: 클래스 분리의 효과

	Step 5 (클래스 없음)	Step 6 (Player 클래스)
이동 로직	GameForm.OnUpdate 안에 직접 작성	player.Update(keys) 한 줄
그리기	GameForm.OnPaint 안에 직접 작성	player.Draw(g) 한 줄
GameForm 역할	입력 + 이동 + 그리기 전부	입력만 받아서 Player에게 전달

GameForm은 “언제 업데이트하고 언제 그릴지”만 결정한다.
 “어떻게 움직이고 어떻게 그릴지”는 Player가 스스로 알고 있다.
 이것이 **역할 분리**이다.

4.2. Step 7: Bullet 클래스 — 총알 발사

총알을 표현하는 Bullet 클래스를 추가한다. Space키를 누르면 발사된다.

Bullet.cs — 새 파일

```

1 using System.Drawing;
2
3 class Bullet
4 {
5     public float X;
6     public float Y;
7     float speed;
8
9     public Bullet(float x, float y, float speed)
10    {
11        X = x;
12        Y = y;
13        this.speed = speed;
14    }
15
16    public void Update()
17    {
18        Y += speed;
19    }
20
21    public bool IsOffScreen()
22    {
23        return Y < -10 || Y > 610;
24    }
25
26    public void Draw(Graphics g, Brush color)
27    {
28        g.FillRectangle(color, X - 2, Y, 4, 10);
29    }

```

30 }

GameForm.cs — 총알 리스트 관리

```
1 class GameForm : Form
2 {
3     Player player = new Player(400, 520);
4     List<Bullet> bullets = new List<Bullet>();
5     HashSet<Keys> keys = new HashSet<Keys>();
6     Timer timer;
7
8     // ... (constructor same as Step 6) ...
9
10    void OnUpdate(object sender, EventArgs e)
11    {
12        player.Update(keys);
13
14        for (int i = 0; i < bullets.Count; i++)
15            bullets[i].Update();
16
17        // Remove off-screen bullets (back to front)
18        for (int i = bullets.Count - 1; i >= 0; i--)
19        {
20            if (bullets[i].IsOffScreen())
21                bullets.RemoveAt(i);
22        }
23
24        Invalidate();
25    }
26
27    protected override void OnKeyDown(KeyEventArgs e)
28    {
29        keys.Add(e.KeyCode);
30
31        if (e.KeyCode == Keys.Space)
32            bullets.Add(new Bullet(
33                player.X, player.Y - 20, -8));
34    }
35
36    // ... (OnKeyUp same as Step 6) ...
37
38    protected override void OnPaint(PaintEventArgs e)
39    {
40        Graphics g = e.Graphics;
41        player.Draw(g);
42
43        for (int i = 0; i < bullets.Count; i++)
44            bullets[i].Draw(g, Brushes.Yellow);
45    }
46 }
```


총알의 생명 주기

1. **생성**: Space키를 누르면 `new Bullet(player.X, player.Y - 20, -8)`로 생성하여 리스트에 추가. 속도 -8은 위쪽으로 이동을 의미한다.
2. **이동**: 매 프레임 `Update()`에서 Y좌표에 `speed`를 더함
3. **제거**: `IsOffScreen()`이 `true`이면 리스트에서 제거

주의: 리스트에서 요소를 삭제할 때는 **뒤에서부터** 삭제해야 한다. 앞에서부터 삭제하면 인덱스가 밀려서 일부 요소를 건너뛸 수 있다.

```
// Wrong: front to back
for (int i = 0; i < list.Count; i++)
    if (...) list.RemoveAt(i); // index shifts!

// Correct: back to front
for (int i = list.Count - 1; i >= 0; i--)
    if (...) list.RemoveAt(i); // safe
```

실습 3: 클래스 분리

1. Step06을 실행하고, 방향키로 비행기를 움직여 보자.
2. Step07을 실행하고, Space키로 총알을 발사해 보자.
3. Bullet의 `speed`를 -4로 바꿔 느린 총알을 만들어 보자.
4. 총알 색상을 `Brushes.Red`로 바꿔 보자. 수정할 곳이 어디인지 찾아보자.

5. 슈팅 게임 완성: Step 8–10

5.1. Step 8: Enemy 클래스 — 적 등장

적을 표현하는 Enemy 클래스를 추가한다. 적은 화면 위쪽에서 자동 생성되어 아래로 내려 오며, 가끔 총알을 발사한다.

Enemy.cs — 새 파일

```
1 using System;
2 using System.Drawing;
3
4 class Enemy
5 {
6     public float X;
7     public float Y;
8     float speed = 2;
9
10    public Enemy(float x, float y)
11    {
12        X = x;
13        Y = y;
14    }
15
16    public void Update()
17    {
18        Y += speed;
19    }
20
21    public bool HitBy(Bullet b)
22    {
23        return Math.Abs(b.X - X) < 20
24            && Math.Abs(b.Y - Y) < 20;
25    }
26
27    public void Draw(Graphics g)
28    {
29        // Inverted triangle (enemy facing down)
30        PointF[] body = new PointF[]
31        {
32            new PointF(X, Y + 20),
33            new PointF(X - 15, Y - 10),
34            new PointF(X + 15, Y - 10)
35        };
36        g.FillPolygon(Brushes.Red, body);
37    }
38 }
```

충돌 판정: HitBy 메서드

HitBy(Bullet b)는 총알과 적의 거리가 20 이내인지 확인한다.

```
return Math.Abs(b.X - X) < 20
    && Math.Abs(b.Y - Y) < 20;
```

이것은 AABB(Axis-Aligned Bounding Box) 충돌 판정이다. 정확한 원형 충돌보다 단순하지만, 게임에서 가장 많이 쓰이는 방식이다.

“적이 총알에 맞았는가?”를 Enemy 클래스가 스스로 판단한다. 이것이 OOP의 장점 — 관련 로직이 해당 클래스 안에 있다.

GameForm.cs — 적 생성과 충돌 처리

```
1 // New fields
2 List<Enemy> enemies = new List<Enemy>();
3 List<Bullet> enemyBullets = new List<Bullet>();
4 Random rand = new Random();
5 int tick = 0;
6
7 void OnUpdate(object sender, EventArgs e)
8 {
9     tick++;
10    player.Update(keys);
11
12    // Move all bullets
13    for (int i = 0; i < bullets.Count; i++)
14        bullets[i].Update();
15    for (int i = 0; i < enemyBullets.Count; i++)
16        enemyBullets[i].Update();
17
18    RemoveOffScreen(bullets);
19    RemoveOffScreen(enemyBullets);
20
21    // Spawn enemy (every ~1 second)
22    if (tick % 60 == 0)
23        enemies.Add(new Enemy(
24            rand.Next(50, 750), -20));
25
26    // Move enemies + shoot
27    for (int i = 0; i < enemies.Count; i++)
28    {
29        enemies[i].Update();
30        if (rand.Next(100) < 2)
31            enemyBullets.Add(new Bullet(
32                enemies[i].X,
33                enemies[i].Y + 20, 4));
34    }
35
36    // Collision: player bullets vs enemies
37    CheckBulletEnemyCollision();
```

```

38
39 // Remove off-screen enemies
40 for (int i = enemies.Count - 1; i >= 0; i--)
41 {
42     if (enemies[i].Y > 650)
43         enemies.RemoveAt(i);
44 }
45
46 Invalidate();
47 }

```

충돌 처리 메서드

```

1 void CheckBulletEnemyCollision()
2 {
3     for (int i = bullets.Count - 1; i >= 0; i--)
4     {
5         for (int j = enemies.Count - 1; j >= 0; j--)
6         {
7             if (enemies[j].HitBy(bullets[i]))
8             {
9                 enemies.RemoveAt(j);
10                bullets.RemoveAt(i);
11                break;
12            }
13        }
14    }
15 }
16
17 void RemoveOffScreen(List<Bullet> list)
18 {
19     for (int i = list.Count - 1; i >= 0; i--)
20     {
21         if (list[i].IsOffScreen())
22             list.RemoveAt(i);
23     }
24 }

```

같은 Bullet 클래스를 적 총알에도 재사용

Bullet 클래스의 speed가 음수이면 위로, 양수이면 아래로 이동한다.

- 플레이어 총알: new Bullet(x, y, -8) — 위로 이동
- 적 총알: new Bullet(x, y, 4) — 아래로 이동

같은 클래스를 생성자 인자만 바꿔서 **재사용**하고 있다. 이것이 클래스 설계의 장점이다.

5.2. Step 9: Hud 클래스 — 점수와 생명 표시

Hud 클래스를 추가하여 점수와 생명을 화면에 표시한다.

Hud.cs — 새 파일

```

1 using System.Drawing;
2
3 class Hud
4 {
5     public int Score = 0;
6     public int Lives = 3;
7     Font font = new Font("Consolas", 16,
8         FontStyle.Bold);
9
10    public void Draw(Graphics g)
11    {
12        g.DrawString("SCORE: " + Score,
13            font, Brushes.White, 10, 10);
14        g.DrawString("LIVES: " + Lives,
15            font, Brushes.White, 10, 35);
16    }
17 }
```

GameForm.cs — Hud 사용

Hud 객체를 필드로 추가하고, 충돌 시 점수/생명을 변경한다.

```

1 Hud hud = new Hud();
2
3 // In CheckBulletEnemyCollision():
4 // When enemy is hit:
5 hud.Score += 100;
6
7 // When player is hit by enemy bullet:
8 hud.Lives--;
9
10 // In OnPaint():
11 hud.Draw(g);
```

DrawString — 화면에 텍스트 출력

`g.DrawString(text, font, brush, x, y)`는 화면에 문자열을 그린다.

- Font 객체로 글꼴, 크기, 스타일을 지정
- 게임의 HUD(Head-Up Display)에 점수, 생명, 상태 메시지 등을 표시할 때 사용

5.3. Step 10: 슈팅 게임 완성

마지막 단계에서는 다음 기능을 추가하여 게임을 완성한다:

- **Game Over** — 생명이 0이 되면 게임 종료 화면 표시
- **재시작** — R키를 누르면 게임 재시작
- **난이도 증가** — 시간이 지날수록 적이 빨라지고 자주 등장
- **적기 충돌** — 적과 플레이어가 직접 부딪혀도 생명 감소
- **배경 별** — 흐르는 별 효과

Player.cs — HitByEnemy 메서드 추가

```

1 public bool HitByEnemy(Enemy en)
2 {
3     return Math.Abs(en.X - X) < 25
4         && Math.Abs(en.Y - Y) < 25;
5 }

```

Hud.cs — DrawCentered 메서드 추가

```

1 public void DrawCentered(Graphics g, string text,
2     float y, float size, Brush color)
3 {
4     Font f = new Font("Consolas", size,
5         FontStyle.Bold);
6     SizeF sz = g.MeasureString(text, f);
7     g.DrawString(text, f, color,
8         (800 - sz.Width) / 2, y);
9 }

```

Enemy.cs — speed를 생성자에서 받도록 변경

```

1 public Enemy(float x, float y, float speed)
2 {
3     X = x;
4     Y = y;
5     this.speed = speed;
6 }

```

GameForm.cs — 핵심 변경 사항

```

1 bool gameOver = false;
2
3 void OnUpdate(object sender, EventArgs e)
4 {
5     if (gameOver)
6     {
7         Invalidate();
8         return;
9     }
10 }

```

```

9      }
10
11     // ... (same as Step 9) ...
12
13     // Spawn with increasing difficulty
14     int spawnRate = Math.Max(20, 60 - tick / 200);
15     if (tick % spawnRate == 0)
16     {
17         enemies.Add(new Enemy(
18             rand.Next(50, 750), -20,
19             1.5f + tick / 1000f));
20
21     // Collision: enemy bullets vs player
22     for (int i = enemyBullets.Count - 1;
23         i >= 0; i--)
24     {
25         if (player.HitBy(enemyBullets[i]))
26         {
27             enemyBullets.RemoveAt(i);
28             hud.Lives--;
29             if (hud.Lives <= 0) gameOver = true;
30         }
31     }
32
33     // Collision: enemy vs player
34     for (int i = enemies.Count - 1; i >= 0; i--)
35     {
36         if (player.HitByEnemy(enemies[i]))
37         {
38             enemies.RemoveAt(i);
39             hud.Lives--;
40             if (hud.Lives <= 0) gameOver = true;
41         }
42     }
43 }

```

재시작 처리

```

1  protected override void OnKeyDown(KeyEventArgs e)
2  {
3      keys.Add(e.KeyCode);
4
5      if (gameOver && e.KeyCode == Keys.R)
6      {
7          Restart();
8          return;
9      }
10
11     if (!gameOver && e.KeyCode == Keys.Space)
12     {
13         bullets.Add(new Bullet(
14             player.X, player.Y - 20, -8));
15     }
16 }

```

```
15
16 void Restart()
17 {
18     player = new Player(400, 520);
19     bullets.Clear();
20     enemies.Clear();
21     enemyBullets.Clear();
22     hud.Score = 0;
23     hud.Lives = 3;
24     gameOver = false;
25     tick = 0;
26 }
```

Game Over 화면과 배경 별

```
1 protected override void OnPaint(PaintEventArgs e)
2 {
3     Graphics g = e.Graphics;
4
5     // Background stars
6     SolidBrush starBrush = new SolidBrush(
7         Color.FromArgb(100, 255, 255, 255));
8     for (int i = 0; i < 50; i++)
9     {
10         int sx = (i * 137 + 53) % 800;
11         int sy = (i * 199 + tick + i * 31) % 600;
12         g.FillRectangle(starBrush, sx, sy, 1, 1);
13     }
14
15     if (gameOver)
16     {
17         hud.Draw(g);
18         hud.DrawCentered(g, "GAME OVER",
19             250, 36, Brushes.Red);
20         hud.DrawCentered(g,
21             "FINAL SCORE: " + hud.Score,
22             300, 20, Brushes.White);
23         hud.DrawCentered(g, "Press R to Restart",
24             340, 16, Brushes.Gray);
25         return;
26     }
27
28     player.Draw(g);
29     // ... (draw bullets, enemies, hud) ...
30 }
```


실습 4: 슈팅 게임

1. Step10을 실행하고 게임을 플레이해 보자.
2. 적에게 맞아 Game Over가 되면, R키로 재시작해 보자.
3. 난이도 조절: `spawnRate` 수식을 바꿔 적 생성 빈도를 조절해 보자.
4. 적 총알 속도(4)를 바꿔보고, 게임 난이도 변화를 체험해 보자.

6. 종합 정리

6.1. 전체 클래스 구조

Step 10의 클래스 관계도

```

Program (Main)
|
v
GameForm : Form
|--- Player      (keyboard input -> move, draw)
|--- Bullet      (move, draw, off-screen check)
|--- Enemy       (move, draw, hit check)
|--- Hud         (score, lives, text draw)
|--- Timer       (game loop, ~60fps)

```

GameForm이 모든 객체를 생성하고 관리한다.

각 클래스는 자신의 **데이터**(위치, 속도)와 **행동**(Update, Draw)을 스스로 가지고 있다.

6.2. 단계별 파일 구성 변화

Step	파일 수	추가된 클래스
Step 01-05	2개	GameForm (모든 로직 포함)
Step 06	3개	Player 분리
Step 07	4개	Bullet 추가
Step 08	5개	Enemy 추가
Step 09-10	6개	Hud 추가

파일이 늘어나도 복잡하지 않다

파일 수는 2개에서 6개로 늘었지만, 각 파일은 **하나의 역할**만 담당하므로 오히려 읽기 쉬워진다.

- “플레이어 이동이 이상해요” → Player.cs만 보면 된다.
- “적 총알이 너무 빨라요” → Bullet.cs의 speed만 확인하면 된다.
- “점수 표시 위치를 바꾸고 싶어요” → Hud.cs만 수정하면 된다.

이것이 Lecture 6에서 배운 OOP의 장점 — **역할 분리**와 **유지보수성**이 실제 게임 개발에서도 동일하게 적용된다.

6.3. 이번 강의에서 배운 핵심 개념

개념	이번 강의에서의 적용
Form 상속	class GameForm : Form으로 커스텀 윈도우 생성
OnPaint 오버라이드	화면을 그리는 코드를 작성하는 진입점
Timer 게임 루프	16ms 간격으로 상태 갱신 → Invalidate → OnPaint
키보드 이벤트	OnKeyDown/OnKeyUp + HashSet으로 동시 입력 처리
클래스 분리	Player, Bullet, Enemy, Hud를 각각 별도 파일로 분리
List를 이용한 객체 관리	총알/적을 List에 추가·삭제하여 동적 관리
충돌 판정	Math.Abs를 이용한 거리 기반 AABB 판정

6.4. 게임 루프 패턴 정리

모든 게임의 기본 구조

```
// 1. Input
//    -> OnKeyDown / OnKeyUp

// 2. Update (OnUpdate)
//    -> Move objects
//    -> Check collisions
//    -> Remove dead objects
//    -> Spawn new objects

// 3. Draw (OnPaint)
//    -> Clear screen (BackColor)
//    -> Draw all objects
//    -> Draw HUD
```

이 “입력 → 갱신 → 그리기” 패턴은 Unity, Unreal 등 모든 게임 엔진에서 동일하게 사용된다. Unity의 Update()가 바로 이 OnUpdate에 해당한다.

도전 과제

1. **플레이어 총알 강화**: 총알 크기를 키우거나, 한 번에 2발씩 발사되도록 수정해 보라.
2. **적 패턴 추가**: 적이 좌우로 지그재그로 움직이도록 Enemy.Update()를 수정해 보라.
3. **파워업 아이템**: 새로운 Item 클래스를 만들어, 먹으면 speed가 올라가는 아이템을 추가해 보라.
4. **보스 만들기**: Boss 클래스를 만들어, HP가 있고 여러 번 맞아야 파괴되는 보스를 만들어 보라.